

Relatório 7 - Mergulho nas IDEs Online para Aprendizado de Máquina

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre algumas IDEs que serão essenciais para trabalhos desenvolvidos no laboratório. A tarefa consistia em assistir o vídeo “*Jupyter Notebook Complete Beginner Guide - From Jupyter to Jupyterlab, Google Colab and Kaggle!*”.

2. Desenvolvimento

As IDEs apresentadas durante a aula se dividem em dois tipos, ambientes de execução local e ambientes de execução em nuvem. Cada uma tem seus pontos fracos e fortes, mas depende muito para que o usuário irá utilizar ou o foco do trabalho dele, sendo assim, a escolha desse ambiente impacta de forma direta na produtividade, a capacidade de visualização de dados e a colaboração em projetos.

Iniciando pelas IDEs que são operadas em ambiente de execução local temos:

- **Python Shell e IPython:**

O Python Shell permite a execução de código direto pelo terminal, porém, ele é executado linha por linha, sendo assim, não é muito recomendado para projetos complexos por causa da sua interatividade ser muito baixa.

```
C:\Users\Edryck>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello World')
Hello World
>>> a = 10
>>> b = '10'
>>> print(a*b)
10101010101010101010
>>>
```

Figura 1. Captura de tela do Python Shell rodando no Prompt de Comando, esse eu sei que sai com Ctrl+Z. (autoria própria)

Já o IPython funciona como uma extensão do Python para torna ele mais interativo, ele serviu de base para como os notebooks do Jupyter são construídos.

```
C:\Users\Edryck>IPython
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.38.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: print('Hello World!')
Hello World!

In [2]: # Não sei como sai e nem como colocar varias linhas em um célula

In [3]: # ... Não é com Alt+Enter

In [4]: # Não dá para sair com Ctrl+Z ou Ctrl+C... Vou fechar o terminal então.

In [5]: _
```

Figura 2. Captura de tela do Prompt de Comando rodando o IPython, OBS.: Não sei sair dele. (autoria própria)

- **Ecossistema Jupyter:**

O Project Jupyter oferece duas principais interfaces que funcionam através de um servidor local (na porta 8888 por padrão):

- **Jupyter Notebook:** Ele é um ambiente baseado em células, assim permitindo integrar o código (Code) e documentação (Markdown), sendo ideal para criar documentos técnicos e cursos sobre alguma linguagem (a linguagem Python é a mais popular e é o padrão de instalação, pelo kernel IPython, mas o Jupyter suporta diversas linguagens) onde o código é explicado em tempo real.

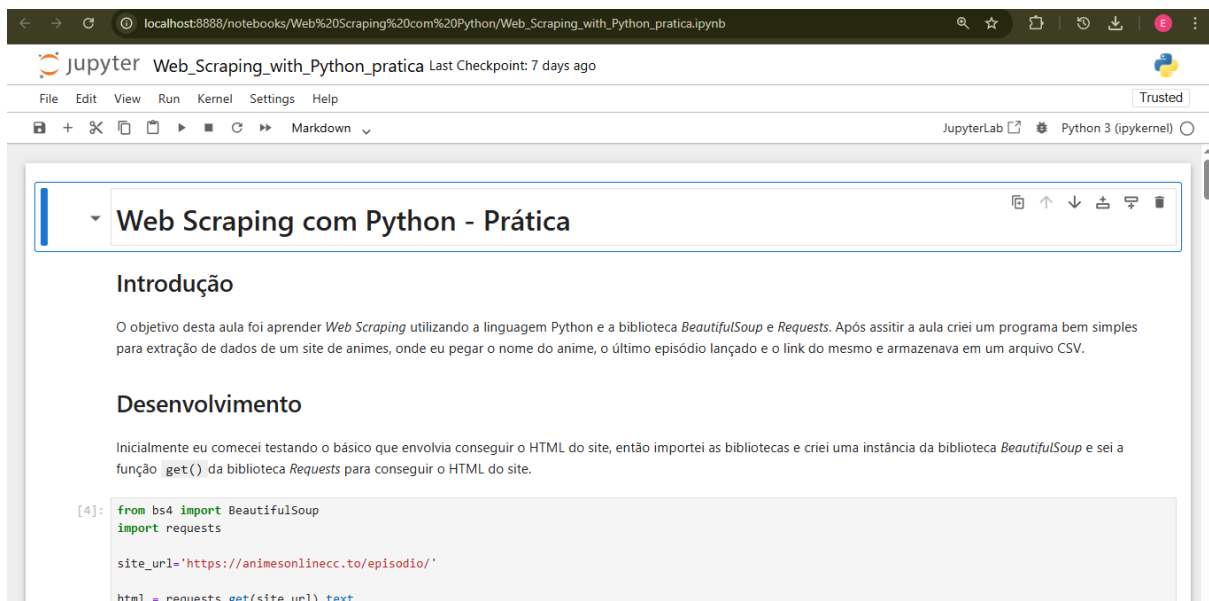


Figura 3. Captura de tela do Jupyter Notebook, é de um notebook do card 8. (autoria própria)

- **JupyterLab:** Ele já é uma interface mais robusta onde inclui navegador de arquivos, gerenciamento de terminais e kernels, e suporta extensões e temas.

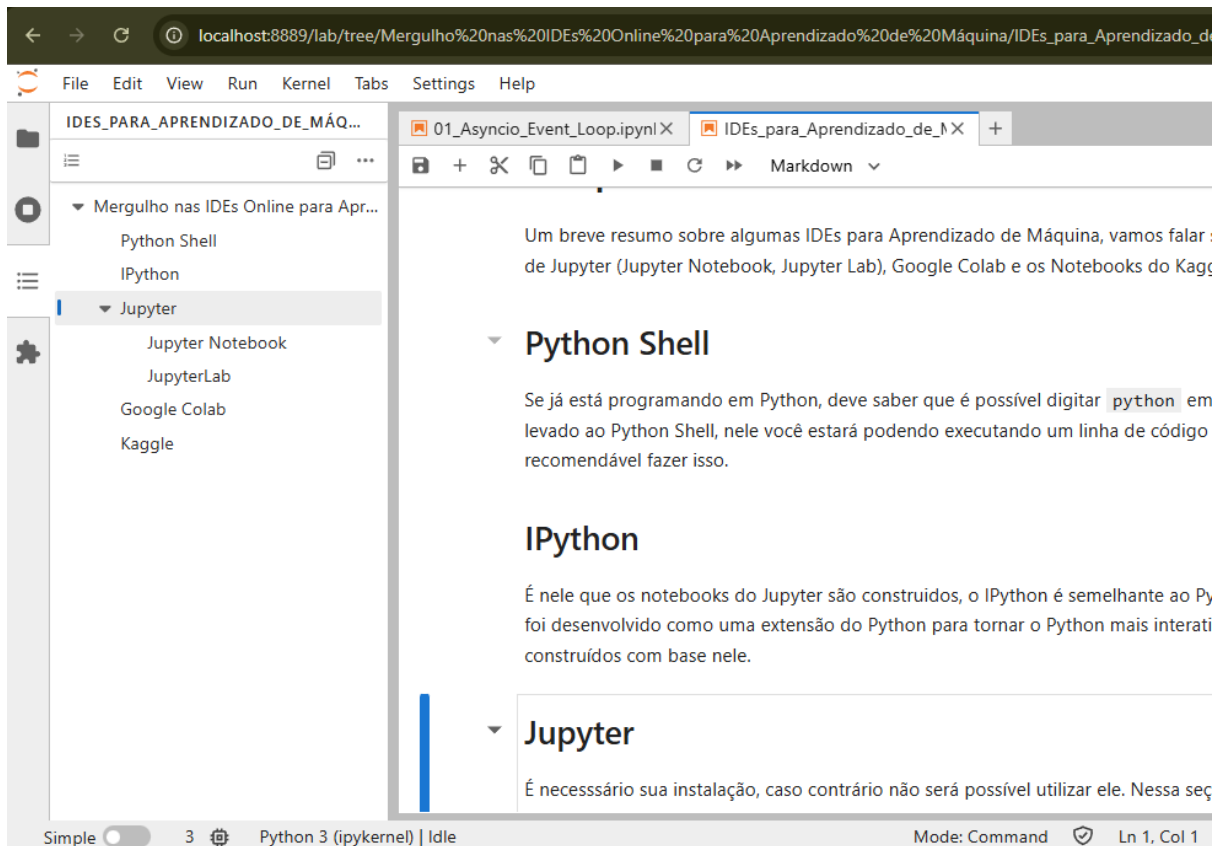


Figura 4. Captura de tela do Jupyter Lab, do notebook que eu estava escrevendo o ipynb desse card. (autoria própria)

Agora sobre as de ambiente de execução em nuvem, após os notebooks se popularizarem, surgiram instâncias em nuvem que tira a necessidade de instalação local de Python e as bibliotecas.

- **Google Colab:**

Ele tem diversas vantagens como permitir a integração direta com o Google Drive e o Github. Ele é bem semelhante aos notebooks do Jupyter em termos de aparência, mas as células de documentação, antes chamadas de “Markdown”, agora são chamadas de “Texto” e esse ambiente dá uma forma de monitorar em tempo real sobre o uso de hardware da instância.

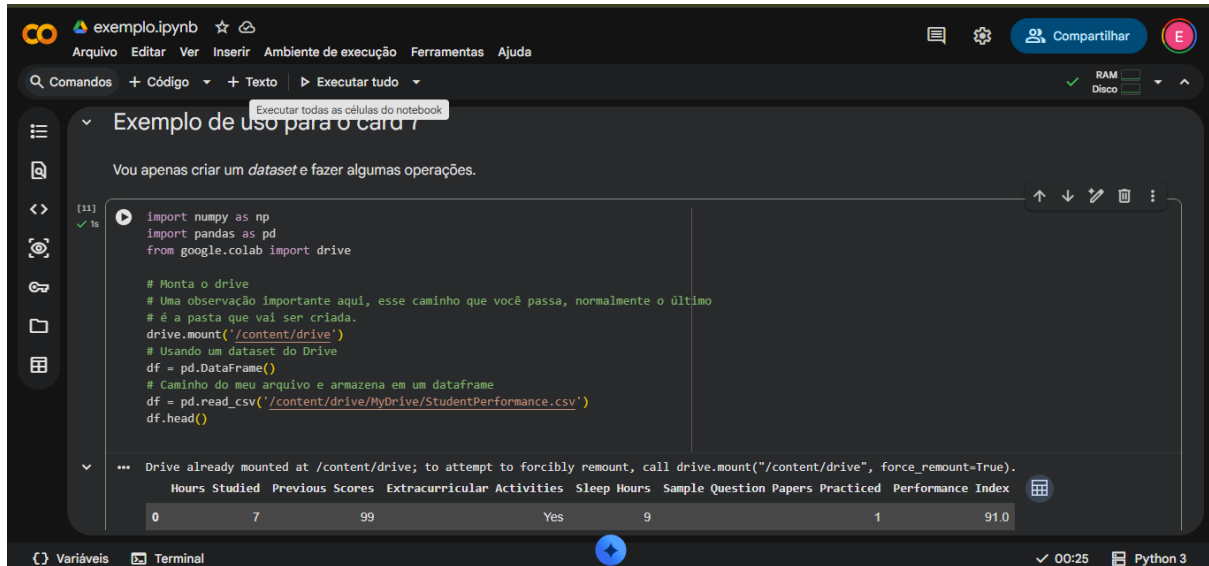


Figura 5. Captura de tela do Google Colab, também importei um arquivo CSV para mostrar a conexão direta com o seu Drive. (autoria própria)

- **Kaggle Notebooks:**

Já o Kaggle Notebooks foca mais em dados, permitindo vincular *datasets* públicos diretamente ao notebook e tem como um diferencial onde exige a execução linear, ou seja, do início ao fim, para salvar versões e permite o usuário compartilhar o código publicamente na plataforma.

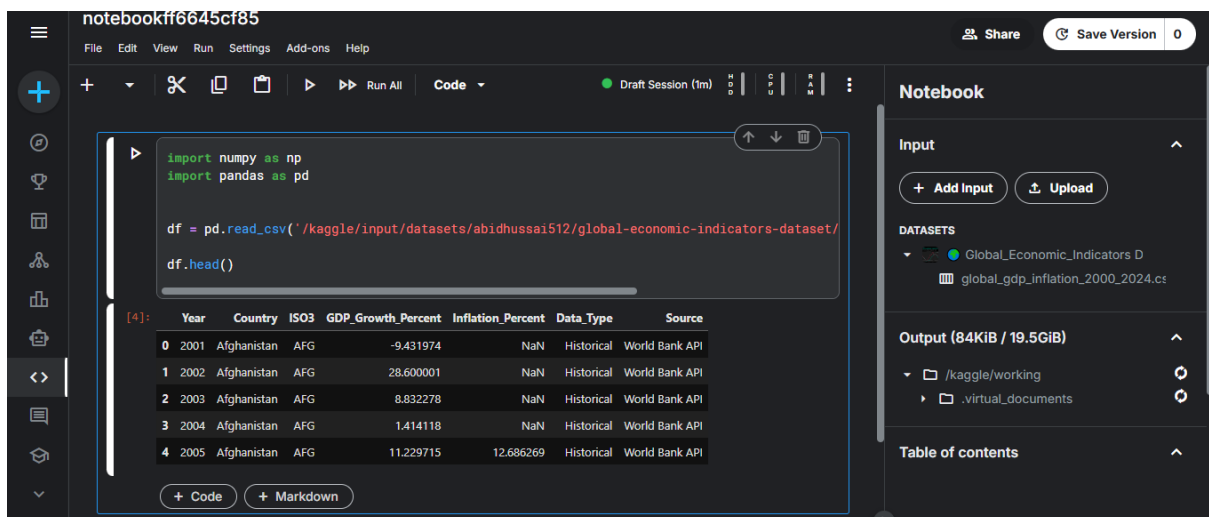


Figura 6. Captura de tela do Kaggle, eu ia usar o mesmo *dataset* do Colab, mas esqueci de salvar ele no Kaggle. (autoria própria)

Uma breve comparação sobre cada IDE:

Recurso	Jupyter	Google Colab	Kaggle
---------	---------	--------------	--------

Instalação Local	É necessário	Não é necessário	Não é necessário
Armazenamento	Disco Local (Normalmente no usuário principal)	Google Drive (Permite <i>download</i> no formato ipynb e py)	Nuvem do Kaggle
Acesso a Datasets	Exige que você baixe os <i>datasets</i>	Disponível via API ou se estiver armazenado no Drive	Nativo, precisa apenas adicionar

Tabela 1. Tabela comparativa de funcionalidade de cada IDE (autoria própria).

O que usar?

Como dito anteriormente, depende muito sobre de qual o foco do trabalho do usuário, no fluxograma abaixo talvez te ajude decidir.

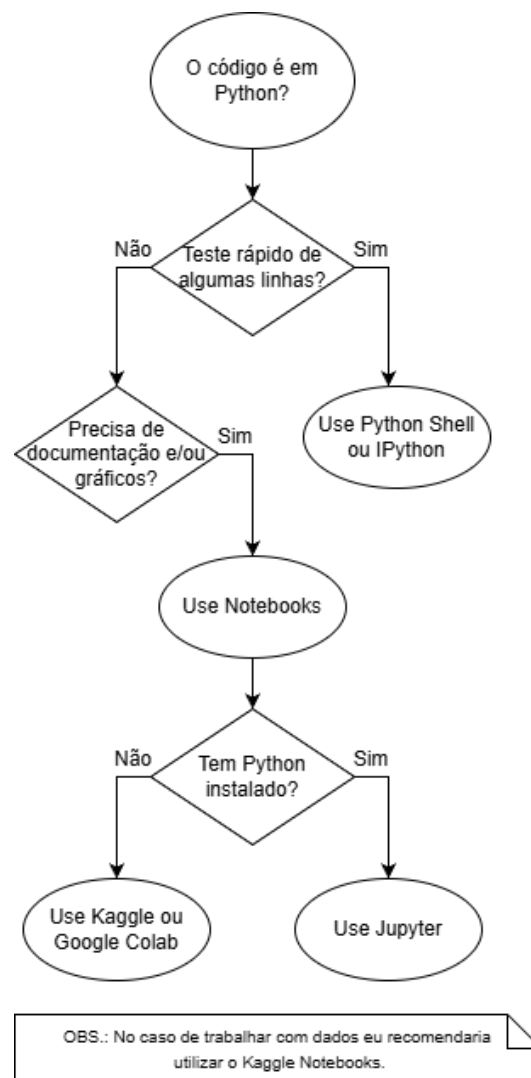


Figura 1. Fluxograma sobre qual IDE usar em determinado caso (autoria própria).

3. Conclusão

Normalmente para quem está aprendendo sobre a linguagem ou para testes rápido, seria interessante utilizar o Python Shell ou o IPython, porém, para o desenvolvimento de modelos que iriam exigir uma documentação e a visualização de gráficos, como os gerados pelo Matplotlib e Seaborn (Seaborn é construído sobre o Matplotlib), o JupyterLab talvez seria a melhor escolha no caso de execução local. Já para projetos que são exigidos a colaboração ou hardware que não está disponível localmente, o Google Colab e o Kaggle oferecem as soluções em nuvem, claro que vem com a vantagem de que vem com as principais bibliotecas já instaladas.

Referências:

Vídeo **Jupyter Notebook Complete Beginner Guide - From Jupyter to Jupyterlab, Google Colab and Kaggle!**:

https://www.youtube.com/watch?v=5pf0_bpNbkw